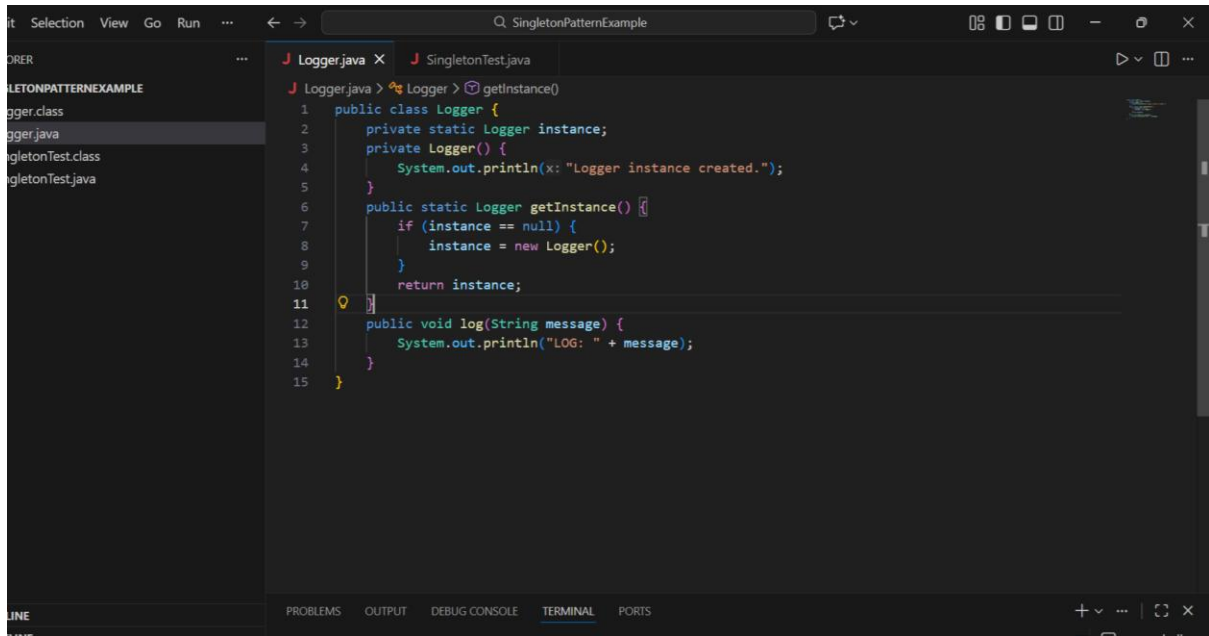


DIGITAL NURTURE 5.0 DEEPSKILLING

WEEK 1 – HANDS-ON

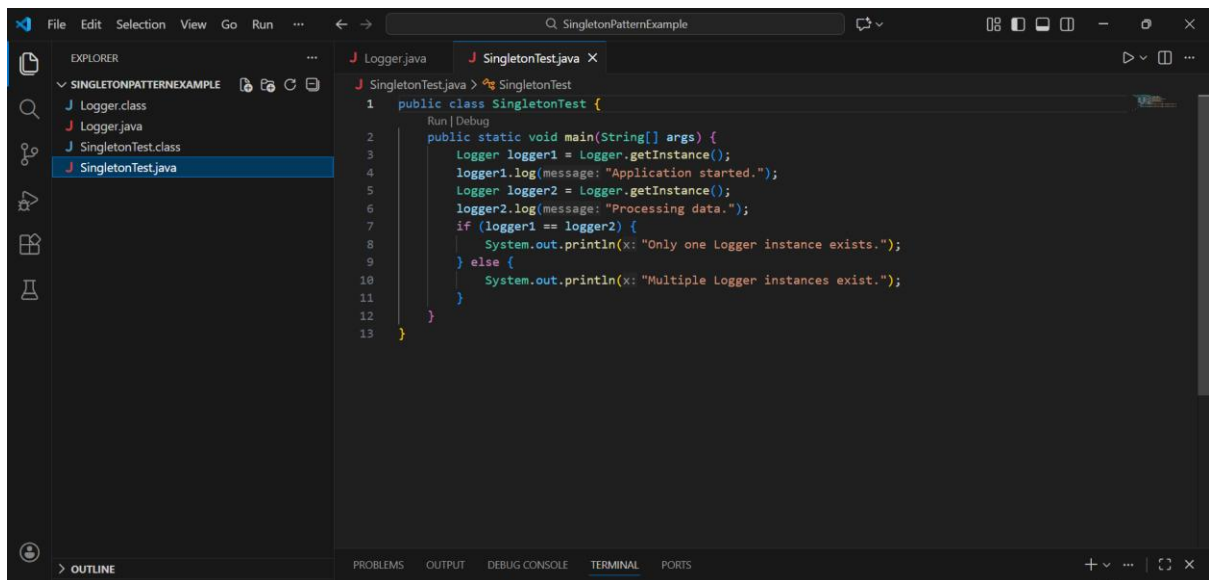
Design principles & Patterns

Exercise 1: Implementing the Singleton Pattern



The screenshot shows an IDE with the file Explorer on the left displaying a project named 'SINGLETONPATTERNEXAMPLE' containing 'Logger.class', 'Logger.java', 'SingletonTest.class', and 'SingletonTest.java'. The main editor window shows 'Logger.java' with the following code:

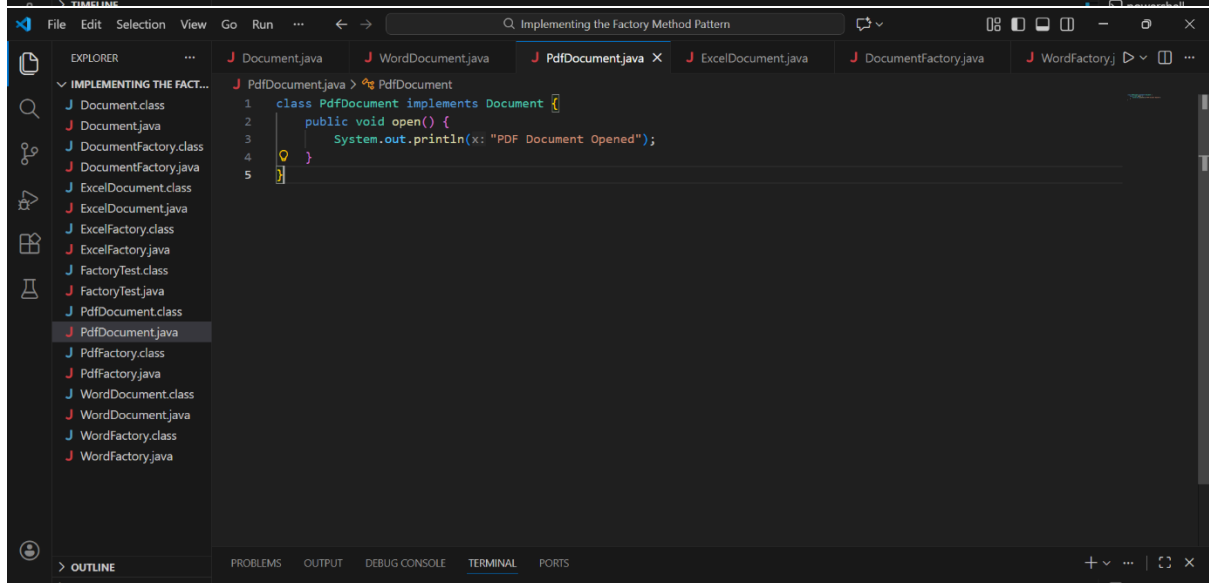
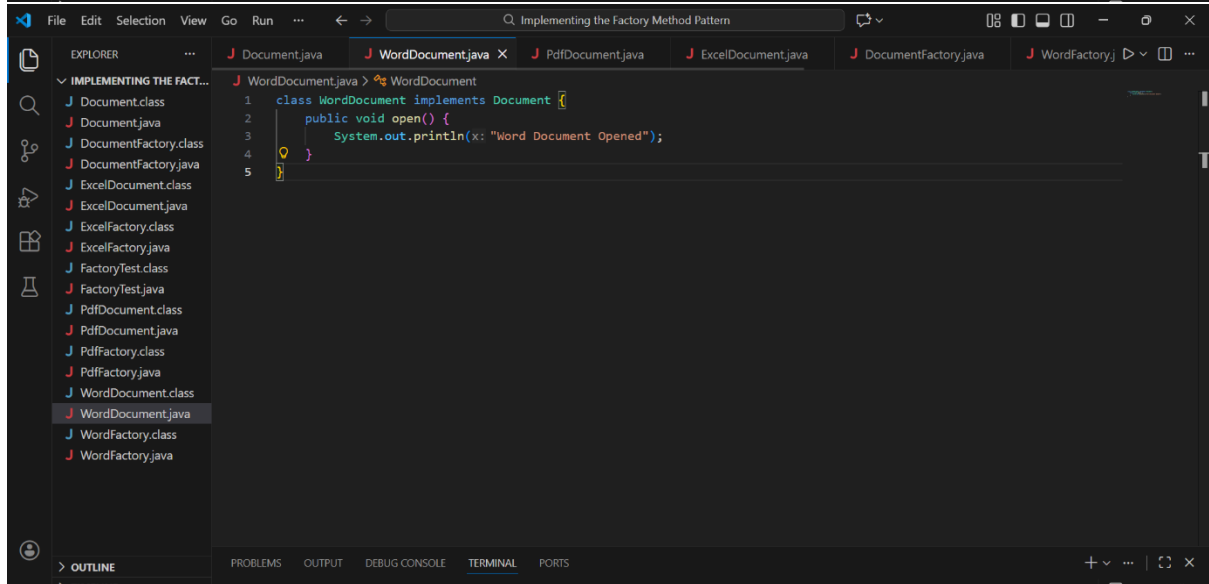
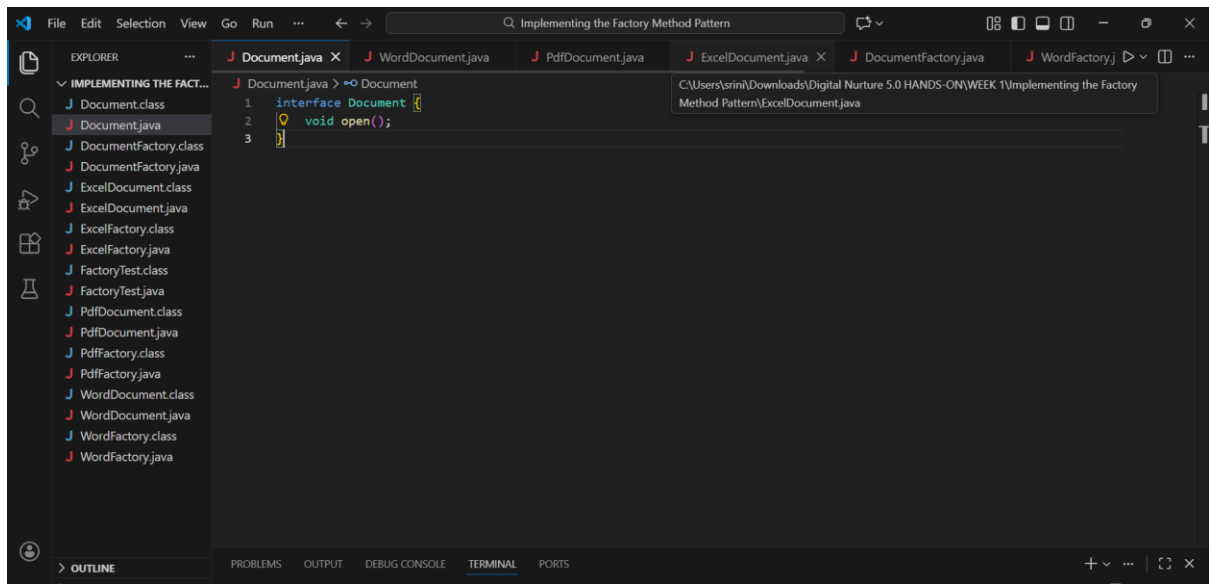
```
1 public class Logger {
2     private static Logger instance;
3     private Logger() {
4         System.out.println(x: "Logger instance created.");
5     }
6     public static Logger getInstance() {
7         if (instance == null) {
8             instance = new Logger();
9         }
10        return instance;
11    }
12    public void log(String message) {
13        System.out.println("LOG: " + message);
14    }
15 }
```



The screenshot shows the same IDE with 'SingletonTest.java' selected in the Explorer and open in the editor. The code in 'SingletonTest.java' is as follows:

```
1 public class SingletonTest {
2     public static void main(String[] args) {
3         Logger logger1 = Logger.getInstance();
4         logger1.log(message: "Application started.");
5         Logger logger2 = Logger.getInstance();
6         logger2.log(message: "Processing data.");
7         if (logger1 == logger2) {
8             System.out.println(x: "Only one Logger instance exists.");
9         } else {
10            System.out.println(x: "Multiple Logger instances exist.");
11        }
12    }
13 }
```

Exercise 2: Implementing the Factory Method Pattern



This screenshot shows the IDE with the `ExcelDocument.java` file open. The Explorer on the left lists the project files under the heading "IMPLEMENTING THE FACT...". The main editor displays the following code:

```
1 class ExcelDocument implements Document {
2     public void open() {
3         System.out.println("Excel Document Opened");
4     }
5 }
6
```

The bottom of the IDE shows tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS.

This screenshot shows the IDE with the `DocumentFactory.java` file open. The Explorer on the left lists the project files. The main editor displays the following code:

```
1 abstract class DocumentFactory {
2     abstract Document createDocument();
3 }
```

The bottom of the IDE shows tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS.

This screenshot shows the IDE with the `WordFactory.java` file open. The Explorer on the left lists the project files. The main editor displays the following code:

```
1 class WordFactory extends DocumentFactory {
2     Document createDocument() {
3         return new WordDocument();
4     }
5 }
```

The bottom of the IDE shows tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS.

File Edit Selection View Go Run ... Q Implementing the Factory Method Pattern

EXPLORER

- Document.class
- Document.java
- DocumentFactory.class
- DocumentFactory.java
- ExcelDocument.class
- ExcelDocument.java
- ExcelFactory.class
- ExcelFactory.java
- FactoryTest.class
- FactoryTest.java
- PdfDocument.class
- PdfDocument.java
- PdfFactory.class
- PdfFactory.java
- WordDocument.class
- WordDocument.java
- WordFactory.class
- WordFactory.java

PDFFactory.java > PdfFactory

```
1 class PdfFactory extends DocumentFactory {
2     Document createDocument() {
3         return new PdfDocument();
4     }
5 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

File Edit Selection View Go Run ... Q Implementing the Factory Method Pattern

EXPLORER

- Document.class
- Document.java
- DocumentFactory.class
- DocumentFactory.java
- ExcelDocument.class
- ExcelDocument.java
- ExcelFactory.class
- ExcelFactory.java
- FactoryTest.class
- FactoryTest.java
- PdfDocument.class
- PdfDocument.java
- PdfFactory.class
- PdfFactory.java
- WordDocument.class
- WordDocument.java
- WordFactory.class
- WordFactory.java

ExcelFactory.java > ExcelFactory

```
1 class ExcelFactory extends DocumentFactory {
2     Document createDocument() {
3         return new ExcelDocument();
4     }
5 }
6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

File Edit Selection View Go Run ... Q Implementing the Factory Method Pattern

EXPLORER

- Document.class
- Document.java
- DocumentFactory.class
- DocumentFactory.java
- ExcelDocument.class
- ExcelDocument.java
- ExcelFactory.class
- ExcelFactory.java
- FactoryTest.class
- FactoryTest.java
- PdfDocument.class
- PdfDocument.java
- PdfFactory.class
- PdfFactory.java
- WordDocument.class
- WordDocument.java
- WordFactory.class
- WordFactory.java

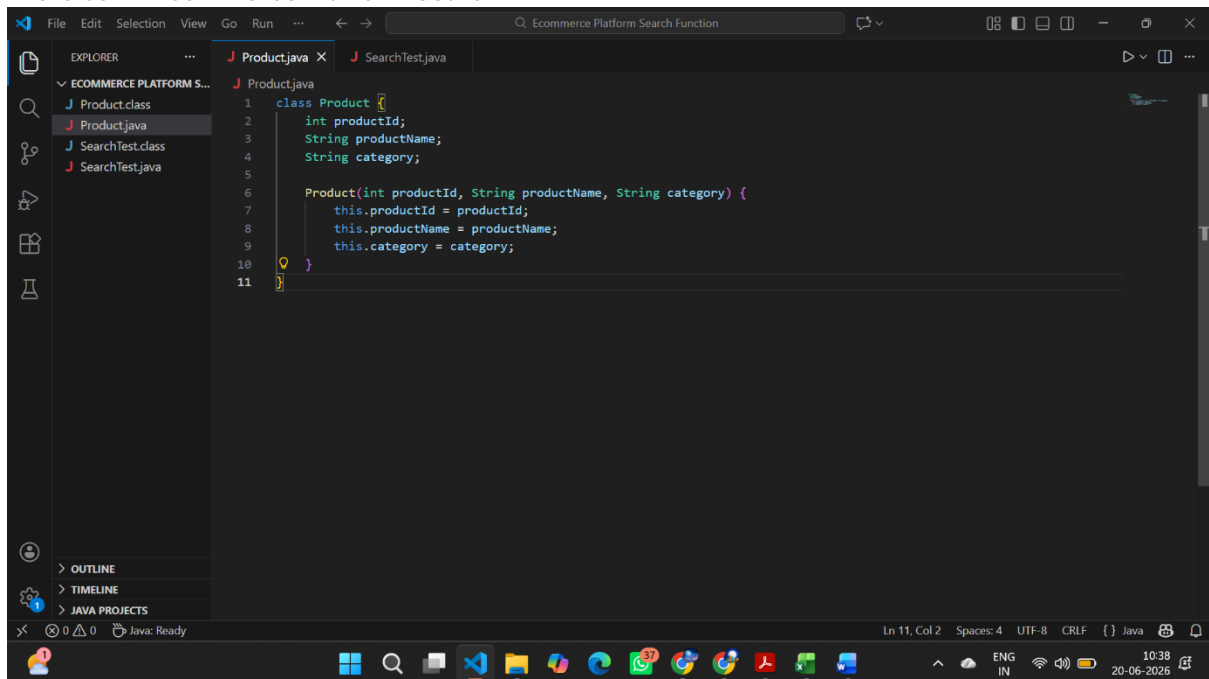
FactoryTest.java > FactoryTest

```
1 public class FactoryTest {
2     Run | Debug
3     public static void main(String[] args) {
4
5         DocumentFactory f1 = new WordFactory();
6         Document d1 = f1.createDocument();
7         d1.open();
8
9         DocumentFactory f2 = new PdfFactory();
10        Document d2 = f2.createDocument();
11        d2.open();
12
13        DocumentFactory f3 = new ExcelFactory();
14        Document d3 = f3.createDocument();
15        d3.open();
16    }
17 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

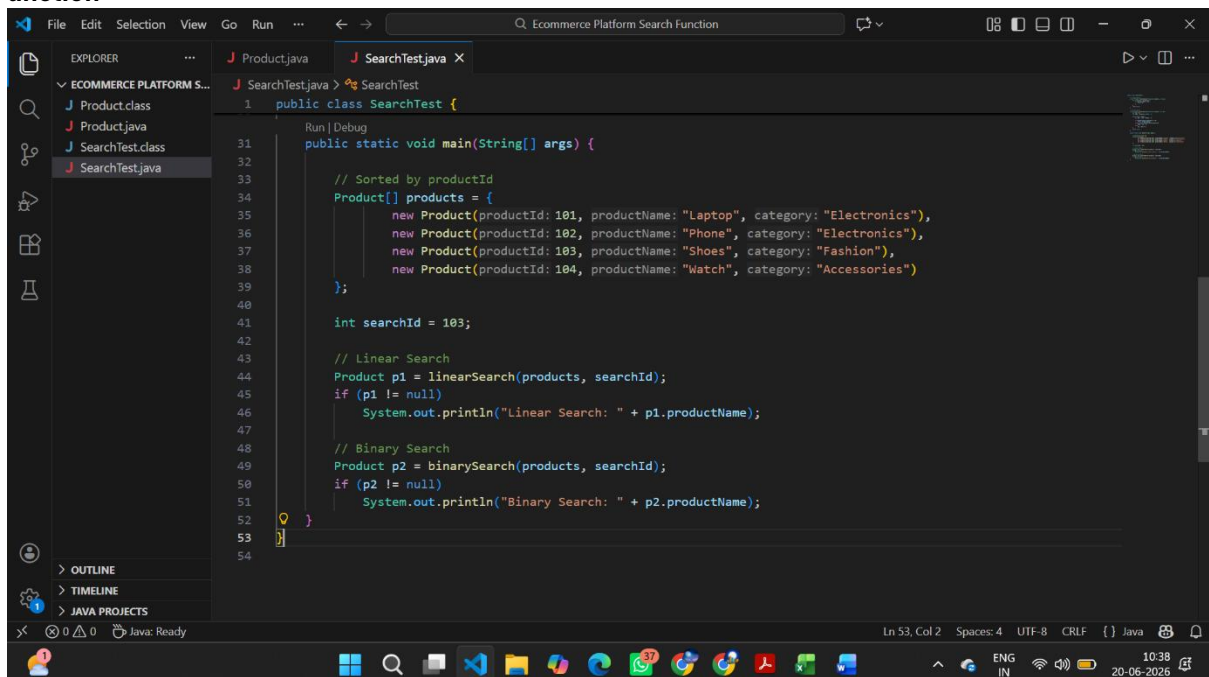
Data structures and Algorithms

Exercise 2: E-commerce Platform Search F



```
1 class Product {
2     int productId;
3     String productName;
4     String category;
5
6     Product(int productId, String productName, String category) {
7         this.productId = productId;
8         this.productName = productName;
9         this.category = category;
10    }
11 }
```

unction



```
1 public class SearchTest {
2
3     public static void main(String[] args) {
4
5         // Sorted by productId
6         Product[] products = {
7             new Product(productId: 101, productName: "Laptop", category: "Electronics"),
8             new Product(productId: 102, productName: "Phone", category: "Electronics"),
9             new Product(productId: 103, productName: "Shoes", category: "Fashion"),
10            new Product(productId: 104, productName: "Watch", category: "Accessories")
11        };
12
13        int searchId = 103;
14
15        // Linear Search
16        Product p1 = linearSearch(products, searchId);
17        if (p1 != null)
18            System.out.println("Linear Search: " + p1.productName);
19
20        // Binary Search
21        Product p2 = binarySearch(products, searchId);
22        if (p2 != null)
23            System.out.println("Binary Search: " + p2.productName);
24    }
25 }
```

Exercise 7: Financial Forecasting

The screenshot shows an IDE window titled "Financial Forecasting". The Explorer pane on the left shows a project named "FINANCIAL FORECASTING" with files "FinancialForecast.class" and "FinancialForecast.java". The main editor displays the code for "FinancialForecast.java".

```
1 public class FinancialForecast {
2
3     // Recursive method
4     static double futureValue(double amount, double rate, int years) {
5         if (years == 0)
6             return amount;
7
8         return futureValue(amount * (1 + rate), rate, years - 1);
9     }
10
11     Run | Debug
12     public static void main(String[] args) {
13
14         double amount = 10000; // Initial amount
15         double rate = 0.10;    // 10% growth rate
16         int years = 3;
17
18         double result = futureValue(amount, rate, years);
19
20         System.out.println("Future Value = " + result);
21     }
22 }
```

PL/SQL programming

Exercise 1: Control Structures

Scenario 1:

The screenshot shows a web browser window with the URL "freesql.com/?utm_source=chatgpt.com". The page is titled "FreeSQL" and has a "Worksheet" tab selected. The SQL editor contains the following code:

```
1 CREATE TABLE Customers (
2     CustomerID NUMBER,
3     Name VARCHAR2(30),
4     Age NUMBER,
5     Balance NUMBER,
6     isVIP VARCHAR2(5)
7 );
8
9 CREATE TABLE Loans (
10     LoanID NUMBER,
11     CustomerID NUMBER,
12     InterestRate NUMBER,
13     DueDate DATE
14 );
15
16 INSERT INTO Customers VALUES (1, 'Ravi', 45, 15000, 'NO');
17 INSERT INTO Customers VALUES (2, 'Sita', 30, 8000, 'NO');
18 INSERT INTO Customers VALUES (3, 'Rohan', 70, 20000, 'NO');
19 INSERT INTO Loans VALUES (101, 1, 15, SYSDATE+20);
20 INSERT INTO Loans VALUES (102, 2, 12, SYSDATE+40);
21 INSERT INTO Loans VALUES (103, 3, 11, SYSDATE+15);
22 COMMIT;
23
24 BEGIN
25     FOR c IN (
26         SELECT CustomerID
27         FROM Customers
28         WHERE Age > 60
29     )
30     LOOP
31         UPDATE Loans
32         SET InterestRate = InterestRate - 1
33         WHERE CustomerID = c.CustomerID;
34     END LOOP;
35 COMMIT;
36 END;
37 /
38 SELECT * FROM Loans;
```

FreeSQL Worksheet

```

1 CREATE TABLE Customers (
2   CustomerID NUMBER,
3   Name VARCHAR2(30),
4   Age NUMBER,
5   Balance NUMBER,
6   IsVIP VARCHAR2(5)
7 );

```

Query result

Download Execution time: 0.002 seconds

	LOANID	CUSTOMERID	INTERESTRATE	DUEDATE
1		101	1	-4 7/10/2026, 8:32:31
2		102	2	12 7/30/2026, 8:32:31
3		103	3	-3 7/5/2026, 8:32:31 A
4		101	1	-2 7/10/2026, 8:33:50
5		102	2	12 7/30/2026, 8:33:50
6		103	3	-1 7/5/2026, 8:33:50 A
7		101	1	1 7/10/2026, 8:35:38
8		102	2	12 7/30/2026, 8:35:38
9		103	3	2 7/5/2026, 8:35:38 A
10		101	1	-4 7/10/2026, 8:33:12

About Oracle Contact Us Legal Notices Terms and Conditions Your Privacy Rights Delete Your FreeSQL Account Cookie Preferences

26.5.2 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

Scenario 2

FreeSQL Worksheet

```

1 CREATE TABLE Customers (
2   CustomerID NUMBER,
3   Name VARCHAR2(30),
4   Age NUMBER,
5   Balance NUMBER,
6   IsVIP VARCHAR2(5)
7 );
8 CREATE TABLE Loans (
9   LoanID NUMBER,
10  CustomerID NUMBER,
11  InterestRate NUMBER,
12  DueDate DATE
13 );
14 INSERT INTO Customers VALUES (1, 'Ravi', 45, 15000, 'NO');
15 INSERT INTO Customers VALUES (2, 'Priya', 30, 8000, 'NO');
16 INSERT INTO Customers VALUES (3, 'Ramesh', 70, 20000, 'NO');
17 INSERT INTO Loans VALUES (101, 1, 10, SYSDATE+20);
18 INSERT INTO Loans VALUES (102, 2, 12, SYSDATE+40);
19 INSERT INTO Loans VALUES (103, 3, 11, SYSDATE+15);
20 COMMIT;
21 BEGIN
22   FOR c IN (
23     SELECT CustomerID
24     FROM Customers
25     WHERE Balance > 10000
26   ) LOOP
27     UPDATE Customers
28     SET IsVIP = 'YES'
29     WHERE CustomerID = c.CustomerID;
30   END LOOP;
31 COMMIT;
32 END;
33 /
34 SELECT * FROM Customers;

```

About Oracle Contact Us Legal Notices Terms and Conditions Your Privacy Rights Delete Your FreeSQL Account Cookie Preferences

26.5.2 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

FreeSQL Worksheet

```
1 CREATE TABLE Customers (  
2   CustomerID NUMBER,  
3   Name VARCHAR2(30),  
4   Age NUMBER,  
5   Balance NUMBER,  
6   IsVIP VARCHAR2(5)  
7 );
```

Query result

Download Execution time: 0.002 seconds

	CUSTOMERID	NAME	AGE	BALANCE	ISVIP
1	1	Ravi	65	15000	YES
2	2	Priya	30	8000	NO
3	3	Kumar	70	20000	YES
4	1	Ravi	65	15000	YES
5	2	Priya	30	8000	NO
6	3	Kumar	70	20000	YES
7	1	Ravi	65	15000	YES
8	2	Priya	30	8000	NO
9	3	Kumar	70	20000	YES
10	1	Ravi	65	15000	YES

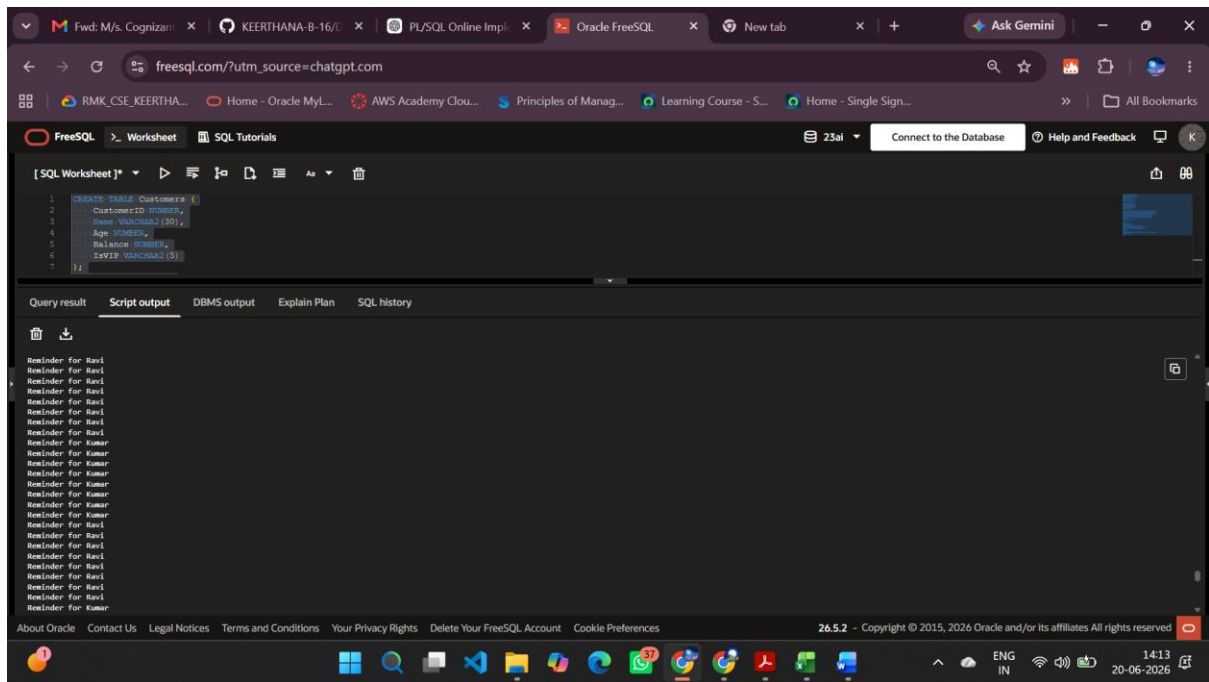
About Oracle Contact Us Legal Notices Terms and Conditions Your Privacy Rights Delete Your FreeSQL Account Cookie Preferences 26.5.2 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

Scenario 3

FreeSQL Worksheet

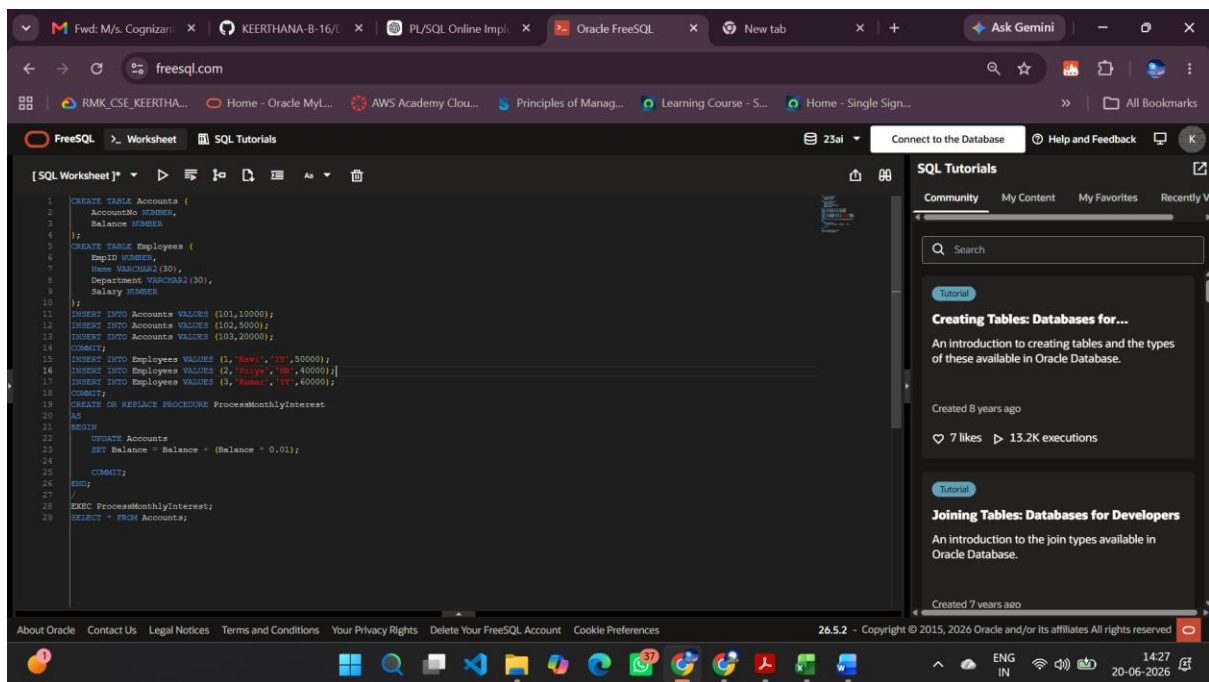
```
1 CREATE TABLE Customers (  
2   CustomerID NUMBER,  
3   Name VARCHAR2(30),  
4   Age NUMBER,  
5   Balance NUMBER,  
6   IsVIP VARCHAR2(5)  
7 );  
8 CREATE TABLE Loans (  
9   LoanID NUMBER,  
10  CustomerID NUMBER,  
11  InterestRate NUMBER,  
12  DueDate DATE  
13 );  
14 INSERT INTO Customers VALUES (1,'Ravi',65,15000,'NO');  
15 INSERT INTO Customers VALUES (2,'Priya',30,8000,'NO');  
16 INSERT INTO Customers VALUES (3,'Kumar',70,20000,'YES');  
17 INSERT INTO Loans VALUES (101,1,10,SYSDATE+20);  
18 INSERT INTO Loans VALUES (102,2,12,SYSDATE+40);  
19 INSERT INTO Loans VALUES (103,3,11,SYSDATE+15);  
20 COMMIT;  
21 SET SERVEROUTPUT ON;  
22  
23 BEGIN  
24   FOR i IN (  
25     SELECT Name  
26     FROM Customers c, Loans l  
27     WHERE c.CustomerID = l.CustomerID  
28     AND l.DueDate <= SYSDATE + 30  
29   )  
30   LOOP  
31     DBMS_OUTPUT.PUT_LINE('Reminder for ' || i.Name);  
32   END LOOP;  
33 END;
```

About Oracle Contact Us Legal Notices Terms and Conditions Your Privacy Rights Delete Your FreeSQL Account Cookie Preferences 26.5.2 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved



Exercise 3: Stored Procedures

Scenario 1



FreeSQL - Worksheet

```

1 CREATE TABLE Accounts (
2   AccountNo NUMBER,
3   Balance NUMBER
4 );
5 CREATE TABLE Employees (
6   EmpID NUMBER,
7   Name VARCHAR2(100),
8 );

```

Query result

Download Execution time: 0.004 seconds

	ACCOUNTNO	BALANCE
1	101	10100
2	102	5050
3	103	20200

SQL Tutorials

Community My Content My Favorites Recently Viewed

Search

Creating Tables: Databases for...

An introduction to creating tables and the types of these available in Oracle Database.

Created 8 years ago

7 likes 13.2K executions

Joining Tables: Databases for Developers

An introduction to the join types available in Oracle Database.

Created 7 years ago

About Oracle Contact Us Legal Notices Terms and Conditions Your Privacy Rights Delete Your FreeSQL Account Cookie Preferences

26.5.2 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

Scenario 2:

FreeSQL - Worksheet

```

1 CREATE TABLE Accounts (
2   AccountNo NUMBER,
3   Balance NUMBER
4 );
5 CREATE TABLE Employees (
6   EmpID NUMBER,
7   Name VARCHAR2(100),
8   Department VARCHAR2(30),
9   Salary NUMBER
10 );
11 INSERT INTO Accounts VALUES (101,10000);
12 INSERT INTO Accounts VALUES (102,5000);
13 INSERT INTO Accounts VALUES (103,20000);
14 COMMIT;
15 INSERT INTO Employees VALUES (1, 'Ravi', 'IT', 50000);
16 INSERT INTO Employees VALUES (2, 'Rishi', 'HR', 40000);
17 INSERT INTO Employees VALUES (3, 'Rohit', 'IT', 60000);
18 COMMIT;
19 CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus
20 (
21   p_dept IN VARCHAR2,
22   p_bonus IN NUMBER
23 )
24 AS
25 BEGIN
26   UPDATE Employees
27   SET Salary = Salary + (Salary * p_bonus / 100)
28   WHERE Department = p_dept;
29   COMMIT;
30 END;
31 /
32 EXEC UpdateEmployeeBonus('IT',10);
33 SELECT * FROM Employees;

```

SQL Tutorials

Community My Content My Favorites Recently Viewed

Search

Creating Tables: Databases for...

An introduction to creating tables and the types of these available in Oracle Database.

Created 8 years ago

7 likes 13.2K executions

Joining Tables: Databases for Developers

An introduction to the join types available in Oracle Database.

Created 7 years ago

About Oracle Contact Us Legal Notices Terms and Conditions Your Privacy Rights Delete Your FreeSQL Account Cookie Preferences

26.5.2 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

FreeSQL - Worksheet

```
1 CREATE TABLE Accounts (
2   AccountNo NUMBER,
3   Balance NUMBER
4 );
5 CREATE TABLE Employees (
6   EmpID NUMBER,
7   Name VARCHAR2(30),
8   Department VARCHAR2(30),
9   Salary NUMBER
10 );
```

Query result

Download Execution time: 0.007 seconds

	EMPID	NAME	DEPARTMENT	SALARY
1	1	Ravi	IT	55000
2	2	Priya	HR	40000
3	3	Kumar	IT	66000
4	1	Ravi	IT	55000
5	2	Priya	HR	40000
6	3	Kumar	IT	66000

SQL Tutorials

Community My Content My Favorites Recently Viewed

Search

Tutorial

Creating Tables: Databases for...

An introduction to creating tables and the types of these available in Oracle Database.

Created 8 years ago

7 likes 13.2K executions

Tutorial

Joining Tables: Databases for Developers

An introduction to the join types available in Oracle Database.

Created 7 years ago

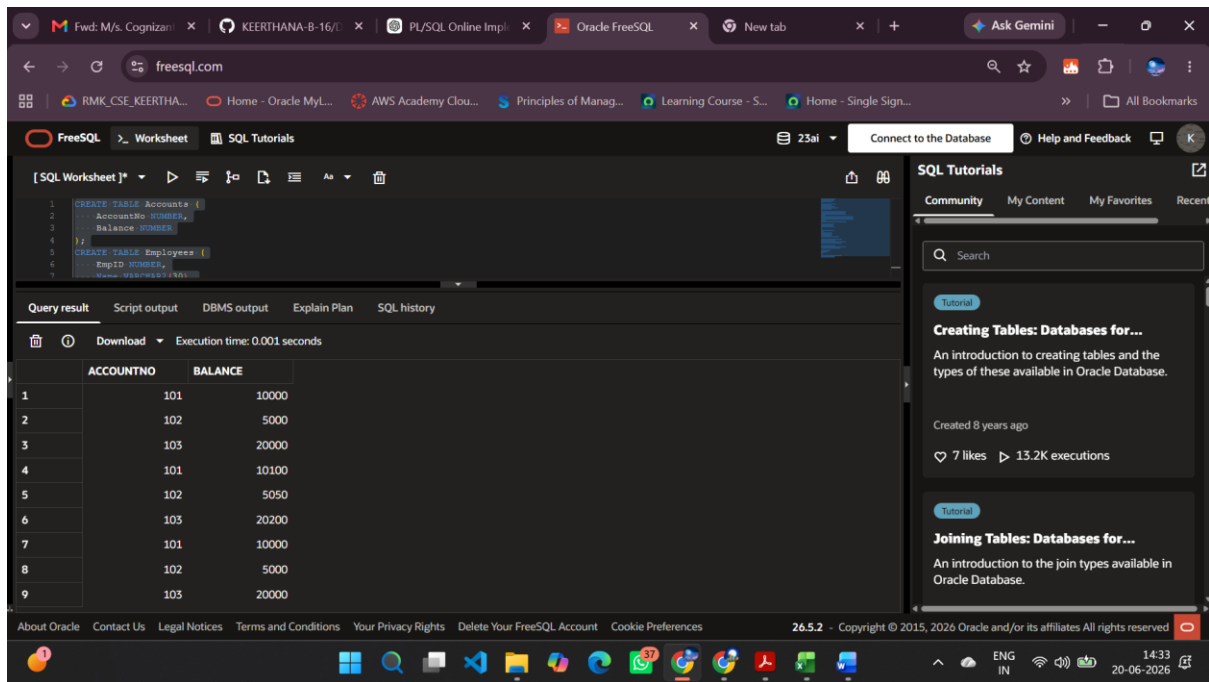
About Oracle Contact Us Legal Notices Terms and Conditions Your Privacy Rights Delete Your FreeSQL Account Cookie Preferences

26.5.2 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

Scenario 3:

FreeSQL - Worksheet

```
1 CREATE TABLE Accounts (
2   AccountNo NUMBER,
3   Balance NUMBER
4 );
5 CREATE TABLE Employees (
6   EmpID NUMBER,
7   Name VARCHAR2(30),
8   Department VARCHAR2(30),
9   Salary NUMBER
10 );
11 INSERT INTO Accounts VALUES (101,10000);
12 INSERT INTO Accounts VALUES (102,5000);
13 INSERT INTO Accounts VALUES (103,20000);
14 COMMIT;
15 INSERT INTO Employees VALUES (1,'Ravi','IT',55000);
16 INSERT INTO Employees VALUES (2,'Priya','HR',40000);
17 INSERT INTO Employees VALUES (3,'Kumar','IT',66000);
18 COMMIT;
19 CREATE OR REPLACE PROCEDURE TransferFunds
20 (
21   p_from IN NUMBER,
22   p_to IN NUMBER,
23   p_amount IN NUMBER
24 )
25 AS
26   v_balance NUMBER;
27 BEGIN
28   SELECT Balance
29   INTO v_balance
30   FROM Accounts
31   WHERE AccountNo = p_from;
32   IF v_balance <= p_amount THEN
33     UPDATE Accounts
34     SET Balance = Balance - p_amount
35     WHERE AccountNo = p_from;
36   ELSE
37     UPDATE Accounts
38     SET Balance = Balance - p_amount
39     WHERE AccountNo = p_to;
40   COMMIT;
41 END IF;
42 END;
43 /
44 EXEC TransferFunds(101,102,2000);
45 SELECT * FROM Accounts;
```



JUnit Testing Exercises

Exercise 1: Setting Up JUnit

pom.xml

```
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
</dependency>
```

CalculatorTest.java

```
import static org.junit.Assert.assertEquals;
import org.junit.Test;

public class CalculatorTest {

    @Test
    public void testAdd() {
        Calculator c = new Calculator();
        assertEquals(5, c.add(2, 3));
    }
}
```

OUTPUT:

Tests run: 1

Failures: 0

Errors: 0

BUILD SUCCESS

Exercise 3: Assertions in JUnit
AssertionsTest.java

```
import static org.junit.Assert.*;

import org.junit.Test;

public class AssertionsTest {

    @Test

    public void testAssertions() {

        // Assert equals

        assertEquals(5, 2 + 3);

        // Assert true

        assertTrue(5 > 3);

        // Assert false

        assertFalse(5 < 3);

        // Assert null

        assertNull(null);

        // Assert not null

        assertNotNull(new Object());

    }

}
```

OUTPUT

Tests run: 1

Failures: 0

Errors: 0

BUILD SUCCESS

Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in Junit

Calculator.java

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

CalculatorTest.java

```
import static org.junit.Assert.assertEquals;  
  
import org.junit.After;  
import org.junit.Before;  
import org.junit.Test;  
  
public class CalculatorTest {  
    private Calculator calculator;  
  
    @Before  
    public void setUp() {  
        calculator = new Calculator();  
        System.out.println("Setup completed");  
    }  
  
    @Test  
    public void testAdd() {  
        // Arrange  
        int a = 2;  
        int b = 3;  
  
        // Act  
        int result = calculator.add(a, b);  
  
        // Assert  
        assertEquals(5, result);  
    }  
  
    @After  
    public void tearDown() {
```

```

        calculator = null;

        System.out.println("Teardown completed");
    }
}

```

OUTPUT

Setup completed

Teardown completed

Tests run: 1

Failures: 0

Errors: 0

BUILD SUCCESS

Mockito exercises

Exercise 1: Mocking and Stubbing

ExternalApi.java

```

public interface ExternalApi {
    String getData();
}

```

MyService.java

```

public class MyService {
    private ExternalApi api;
    public MyService(ExternalApi api) {
        this.api = api;
    }
    public String fetchData() {
        return api.getData();
    }
}

```

MyServiceTest.java

```

import static org.junit.Assert.assertEquals;
import static org.mockito.Mockito.*;
import org.junit.Test;
import org.mockito.Mockito;
public class MyServiceTest {
    @Test
    public void testExternalApi() {
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        when(mockApi.getData()).thenReturn("Mock Data");
        MyService service = new MyService(mockApi);
        String result = service.fetchData();
        assertEquals("Mock Data", result);
    }
}

```

```
}
```

OUTPUT

Tests run: 1
Failures: 0
Errors: 0
BUILD SUCCESS

Exercise 2: Verifying Interactions

ExternalApi.java

```
public interface ExternalApi {  
    String getData();  
}
```

MyService.java

```
public class MyService {  
    private ExternalApi api;  
    public MyService(ExternalApi api) {  
        this.api = api;  
    }  
    public String fetchData() {  
        return api.getData();  
    }  
}
```

MyServiceTest.java

```
import static org.mockito.Mockito.*;  
import org.junit.Test;  
import org.mockito.Mockito;  
public class MyServiceTest {  
    @Test  
    public void testVerifyInteraction() {  
        // Create mock object  
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);  
        // Create service object  
        MyService service = new MyService(mockApi);  
        // Call method  
        service.fetchData();  
        // Verify interaction  
        verify(mockApi).getData();  
    }  
}
```

OUTPUT

Tests run: 1
Failures: 0
Errors: 0
BUILD SUCCESS

SLF4J logging framework

Exercise 1: Logging Error Messages and Warning Levels

pom.xml


```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>1.7.30</version>
</dependency>
```

```
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>1.2.3</version>
</dependency>
```

LoggingExample.java

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class LoggingExample {
    private static final Logger logger =
        LoggerFactory.getLogger(LoggingExample.class);
    public static void main(String[] args) {
        logger.error("This is an error message");
        logger.warn("This is a warning message");
    }
}
```

OUTPUT

```
ERROR LoggingExample - This is an error message
WARN  LoggingExample - This is a warning message
```